Práctica 1
Sistemas Operativos

Luján Noemí Patrone

11 de octubre de 2026

Índice general

1. **Ejercicio 1.a: Árbol de Procesos en Malla**
   - **1.1.** Enunciado del Ejercicio
     - 1.1.1. Estructura de la matriz solicitada.
   - **1.2.** Identificación y Justificación de Tareas
     - 1.2.1. Validación y conversión de argumentos.
     - 1.2.2. Generación matricial de procesos.
     - 1.2.3. Sincronización temporal y control de tiempos de vida.
     - 1.2.4. Ejecución del comando pstree mediante un proceso auxiliar.
     - 1.2.5. Terminación ordenada y recolección de hijos con wait().
   - **1.3.** Diagrama de la Jerarquía de Procesos.
   - **1.4.** Análisis detallado del código fuente
     - 1.4.1. Bloque de validación e inicialización.
     - 1.4.2. Bloque de creación horizontal (Columnas).
     - 1.4.3. Bloque de creación vertical (Filas).
     - 1.4.4. Bloque del proceso raíz: Espera, ejecución de pstree y recogida de hijos.
     - 1.4.5. Bloque de los procesos hijos: Espera con wait(), pausa y salida.

2. **Ejercicio 1.b: Sincronización Reactiva con Señales**
   - **2.1.** Enunciado del Ejercicio
     - 2.1.1. Estructura del árbol solicitada.
     - 2.1.2. Resultado esperado en la terminal.
   - **2.2.** Identificación y Justificación de Tareas
     - 2.2.1. Recepción y validación de argumentos.
     - 2.2.2. Generación de la jerarquía de procesos.
     - 2.2.3. Temporización con alarma en el proceso Z (sin usar sleep).
     - 2.2.4. Notificación entre procesos con señales y ejecución de pstree.
     - 2.2.5. Destrucción ordenada del árbol de abajo hacia arriba.
   - **2.3.** Diagrama de la Jerarquía de Procesos.
   - **2.4.** Análisis detallado del código fuente
     - 2.4.1. Manejadores de señal y funciones auxiliares (tiempo_pasado, despertador, errores).
     - 2.4.2. Función main(): Proceso coordinador ejec.
     - 2.4.3. Función proceso_A(): Espera de señal y disparo de pstree.
     - 2.4.4. Función proceso_B(): Creación del abanico (X, Y, Z) y cascada de muerte.

3. **Ejercicio 2: Fragmentación de Archivos y Tuberías**
   - **3.1.** Enunciado del Ejercicio

3.1.1. Parámetros de entrada y esquema de fragmentación.
3.1.2. Resultado esperado en la terminal.
3.2. Identificación y Justificación de Tareas
3.2.1. Recepción y validación de argumentos.
3.2.2. Inspección del tamaño del archivo y cálculo de partes (stat).
3.2.3. Canal de comunicación mediante tuberías (pipe) y gestión de descriptores.
3.2.4. Estrategia de procesamiento secuencial y sincronización con wait().
3.2.5. Formateo manual de nombres sin funciones de alto nivel.
3.2.6. Transferencia de bloques binarios en bajo nivel (read y write).
3.3. Diagrama de la Jerarquía de Procesos.
3.4. Análisis detallado del código fuente
3.4.1. Bloque de validación y apertura inicial (stat, open).
3.4.2. Bloque del Proceso Padre: Lectura de bloques, envío por tubería y espera.
3.4.3. Bloque del Proceso Hijo: Creación del trozo, recepción y persistencia.

4. Ejercicio 3: Memoria Compartida

4.1. Enunciado del Ejercicio
4.1.1. Estructura del árbol (Tronco y Abanico).
4.2. Identificación y Justificación de Tareas
4.2.1. Recepción y validación de argumentos.
4.2.2. Necesidad y configuración del segmento de memoria compartida (shmget, shmat).
4.2.3. Construcción del tronco vertical de ancestros sin ruptura estructurada.
4.2.4. Creación del abanico horizontal de subhijos y registro concurrente de PIDs.
4.2.5. Sincronización ascendente con wait() y terminación limpia.
4.2.6. Liberación y destrucción del segmento IPC (shmdt, shmctl).
4.3. Diagrama de la Jerarquía de Procesos.
4.4. Análisis detallado del código fuente
4.4.1. Bloque de validación, creación y vinculación de Memoria IPC.
4.4.2. Bloque del Tronco Vertical: Registro de ancestros y descenso lineal.
4.4.3. Bloque del Abanico de Subhijos: Registro y lectura de linaje.
4.4.4. Bloque del Superpadre: Lectura final, muerte y destrucción IPC.

5. Conclusiones

- Comparativa entre los mecanismos de comunicación utilizados (Señales vs. Tuberías vs. Memoria Compartida).
- Consideraciones de diseño tenidas en cuenta: prevención de procesos zombies/huérfanos y gestión responsable de recursos del núcleo.

Capítulo 1

Ejercicio 1.a

1.1. Enunciado del Ejercicio

Realiza un programa llamado malla.c que produzca el siguiente árbol de procesos. El programa recibirá dos argumentos ‘x’ e ‘y’ que representan el número de filas y columnas. Para comprobar la estructura de procesos que realmente estamos creando debemos emplear la llamada al sistema “pstree –c”.

1.1.1. Estructura de la Matriz Solicitada

La jerarquía de procesos que se debe crear del apartado **a** es la siguiente:

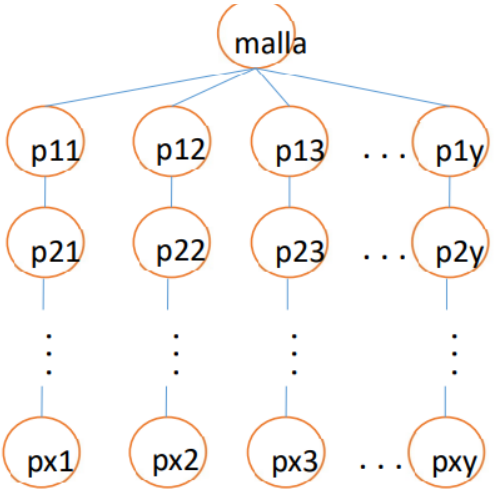


Figura 1.1: Diagrama de la matriz de procesos a generar.

1.2. Identificación y Justificación de Tareas

1.2.1 Validación y Conversión de Argumentos

El enunciado comienza “El programa recibirá dos argumentos 'x' e 'y' que representan el número de filas y columnas”. Esto establece el requisito de que la aplicación debe interactuar con el usuario mediante la línea de comandos. La justificación de esta tarea es la necesidad de configurar dinámicamente en tiempo de ejecución las dimensiones de la matriz de procesos a construir. Para llevar a cabo esto, el programa comprueba inicialmente mediante argc != 3

que el usuario haya facilitado exactamente los dos parámetros requeridos, de lo contrario, muestra un mensaje de error y finaliza inmediatamente con exit(1). A continuación, emplea la función atoi() para convertir las cadenas de caracteres de argv[1] y argv[2] a valores enteros almacenados en las variables filas y columnas. Finalmente, el proceso original obtiene su identificador mediante getpid() y lo almacena en pid_raiz, inicializando la bandera soy_raiz = 1 para permitir distinguir su rol frente al resto de la jerarquía.

1.2.2. Generación Matricial de Procesos

La instrucción central del problema dice: “Realiza un programa llamado malla.c que produzca el siguiente árbol de procesos”. Este requisito impone la construcción controlada de una jerarquía matricial con Y cabeceras de columna y cadenas verticales descendentes de longitud X, evitando a toda costa creación incontrolada de procesos. Para resolver esta estructura, el diseño divide la creación en dos fases consecutivas mediante el valor de retorno de fork(). En primer lugar, la creación horizontal se implementa mediante un bucle limitado por la condición pid > 0, lo que permite que únicamente el proceso raíz engendre a los Y líderes de columna, saliendo cada hijo de forma inmediata al recibir un valor de retorno 0 y marcar soy_raiz = 0. En segundo lugar, la fase vertical restringe el acceso a los procesos descendientes (!soy_raiz) a través de un bucle condicionado a pid == 0, permitiendo que cada eslabón cree a un único hijo hacia abajo y quede a la espera, conformando cadenas estrictamente lineales de X niveles.

1.2.3. Sincronización Temporal y Control de Tiempos de Vida

Aunque no aparece como una orden explícita en el texto, existe el requisito de coordinar los tiempos de vida de los procesos para que toda la malla se mantenga simultáneamente en memoria en el instante de su comprobación. Dado que la creación de procesos es asíncrona, sin un mecanismo de estabilización las hojas del árbol terminarían su ejecución antes de que el núcleo completase los niveles inferiores. La justificación de esta tarea es prevenir la desaparición rápida de ramas de la jerarquía. Para resolverlo, se han implementado dos mecanismos complementarios: el proceso raíz realiza una pausa pasiva inicial mediante sleep(1) para dar el margen necesario de consolidación del árbol, mientras que los procesos situados en la base de cada columna (las hojas, donde pid == 0) se suspenden temporalmente con sleep(3) actuando como anclas para sostener vivas a sus respectivas columnas.

1.2.4. Ejecución del Comando PSTREE Mediante un Proceso Auxiliar

El enunciado impone: “Para comprobar la estructura de procesos que realmente estamos creando debemos emplear la llamada al sistema pstree -c”. La justificación de esta tarea viene en invocar un programa externo del sistema operativo para inspeccionar la tabla de procesos sin destruir el mapa de memoria del proceso coordinador. Dado que la llamada al sistema

execlp() reemplaza de forma irreversible el código del proceso invocador por el nuevo, ejecutarla directamente desde la raíz destruiría su capacidad para gestionar luego el cierre ordenado del árbol. Para solucionar esta limitación, el proceso raíz convierte su identificador a formato de texto mediante sprintf(pid_str, "%d", pid_raiz) y realiza una bifurcación auxiliar con fork(). Es este proceso hijo el que ejecuta execlp("pstree", "pstree", "-c", pid_str, NULL), imprimiendo la jerarquía en pantalla y finalizando su ciclo de vida sin alterar al proceso raíz.

1.2.5. Terminación Ordenada y Recolección de Hijos con WAIT()

Al igual que en las normas generales de gestión de procesos de la práctica, se mantiene el principio de que los procesos padre no deben finalizar antes que sus hijos, garantizando que el sistema no retenga procesos zombies. La justificación es asegurar una desasignación limpia de los recursos del núcleo al terminar el programa. La implementación se forma en dos niveles de recolección: dentro de cada columna, los eslabones intermedios (pid > 0) permanecen detenidos en una llamada wait(NULL) hasta que la terminación del sleep(3) de las hojas desbloquea en cascada la terminación de cada nivel vertical. En la cima de la jerarquía, el proceso raíz ejecuta un bucle con columnas + 1 llamadas a wait(NULL), recolectando de forma estricta tanto al proceso auxiliar encargado de ejecutar pstree como a los Y líderes de columna, concluyendo la ejecución de forma ordenada.

1.3. Diagrama de la Jerarquía de Procesos

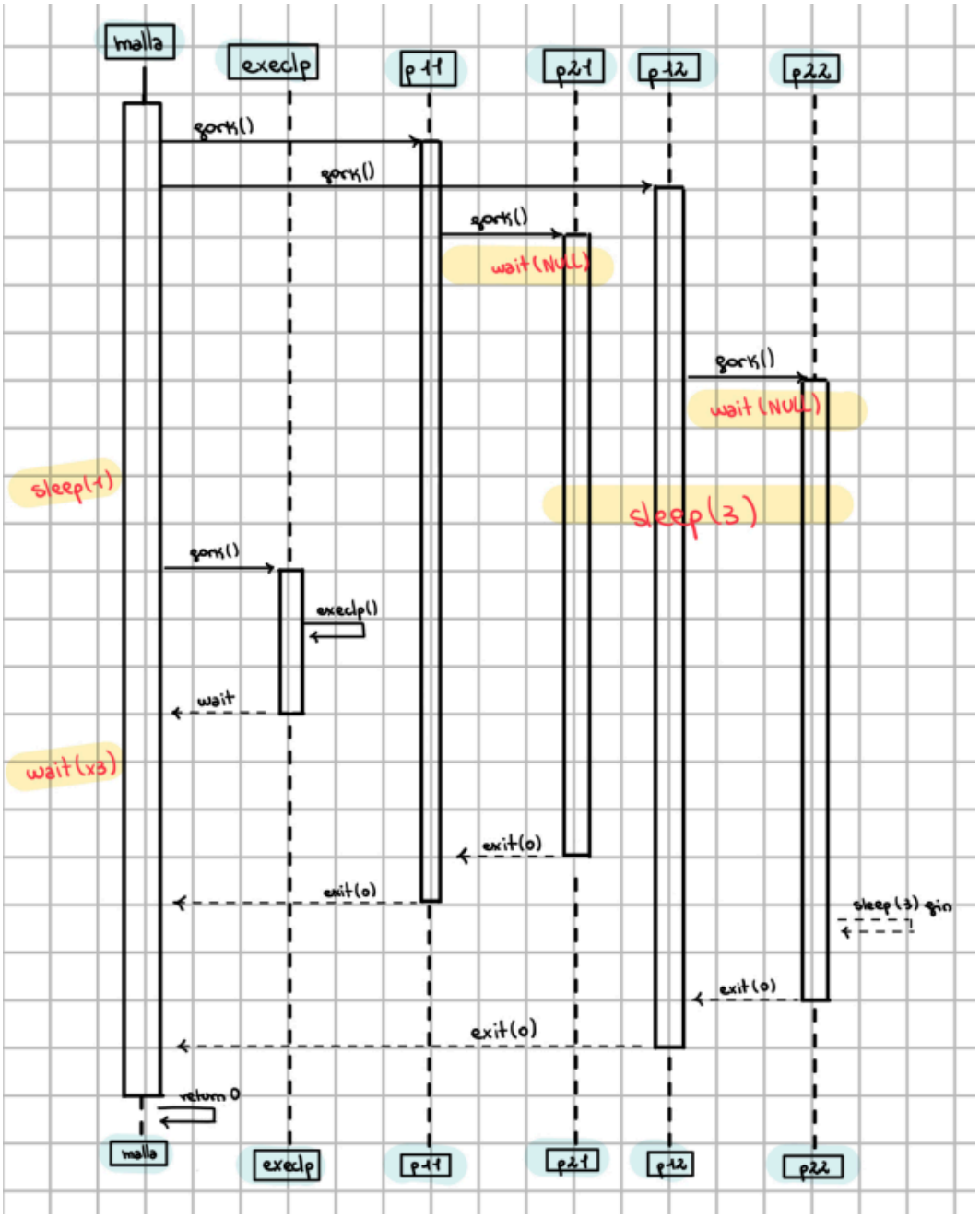


Figura 1.3: Diagrama de secuencia de malla.c

1.4. Análisis Detallado del Código Fuente

1.4.1. Bloque de Validación e Inicialización

El propósito de este bloque inicial es procesar los parámetros recibidos por la línea de comandos, verificar que se cumpla la sintaxis esperada y registrar el estado inicial del proceso raíz antes de comenzar la creación del árbol. Para ello, el programa comprueba mediante la condición argc != 3 que se hayan introducido exactamente dos argumentos adicionales al ejecutable, finalizando la ejecución de forma inmediata con exit(1) en caso de error. A continuación, se utiliza la función atoi() para transformar las cadenas de caracteres de argv[1] y argv[2] en los valores enteros de las variables filas y columnas. Por último, se obtiene el identificador único del proceso raíz mediante la llamada getpid() para almacenarlo en pid_raiz, e inicializa las variables de control soy_raiz = 1 y pid = 1, permitiendo que el proceso coordinador distinga su rol frente al resto de descendientes que nacerán a continuación.

Llamadas al Sistema Utilizadas: exit(), getpid().

```
if (argc != 3) {
    printf("Argumentos incorrectos\n");
    exit(1);
}

int filas = atoi(argv[1]);
int columnas = atoi(argv[2]);
int pid = 1;
int soy_raiz = 1;
pid_t pid_raiz = getpid();
```

1.4.2. Bloque de Creación Horizontal (Columnas)

El objetivo de este bloque es generar la fila superior de la matriz, crear a los líderes de cada una de las columnas solicitadas. Mediante un bucle for condicionado por la expresión i < columnas && pid > 0, se garantiza que únicamente el proceso raíz pueda realizar llamadas a fork(), ya que tras cada bifurcación el padre recibe el PID positivo del hijo recién creado y avanza a la siguiente iteración hasta completar las Y columnas. Por el contrario, en el instante en que nace cada proceso hijo, la llamada fork() le devuelve el valor 0, lo que invalida de inmediato la condición de permanencia del bucle, haciendo que marque la bandera soy_raiz = 0 y salga del bucle en el acto sin engendrar hermanos adicionales a lo ancho.

Llamadas al Sistema Utilizadas: fork().

```c
for (int i = 0; i < columnas && pid > 0; i++) {
    pid = fork();
    if (pid == 0) {
        soy_raiz = 0;
    }
}
```

1.4.3. Bloque de Creación Vertical (Filas)

La finalidad de este bloque es desplegar las cadenas descendentes que cuelgan de cada líder de columna para completar la cantidad de X filas de la matriz. La condición inicial if (!soy_raiz) restringe el acceso a este bucle exclusivamente a los descendientes generados en la fase horizontal previa. Dentro del bucle, la condición de permanencia j < filas - 1 && pid == 0 logra que cada eslabón cree a un único hijo hacia abajo con fork(), ya que el proceso padre recibe un valor positivo (pid > 0) y abandona el bucle para quedarse a la espera de su descendiente, mientras que el hijo recién creado recibe pid == 0, avanza a la siguiente iteración y repite el procedimiento. Este comportamiento se repite exactamente X - 1 veces por columna, trazando una cadena vertical estricta de X procesos.

Llamadas al Sistema Utilizadas: fork().

```c
if (!soy_raiz) {
    for (int j = 0; j < filas - 1 && pid == 0; j++) {
        pid = fork();
    }
}
```

1.4.4. Bloque del Proceso Raíz: Espera, Ejecución y Recogida de Hijos

Este bloque define el comportamiento del proceso raíz coordinador una vez terminada la etapa de creación de la matriz. En primer lugar, la raíz realiza una pausa con sleep(1) para dar tiempo a que todos los procesos de los niveles inferiores terminen de crearse y estabilizarse en la tabla de procesos del sistema operativo. Luego, convierte su identificador numérico a texto mediante sprintf() y genera a un proceso hijo auxiliar con fork() para delegar en él la ejecución del comando de verificación, evitando así que la llamada execlp() sobrescriba la imagen de memoria de la raíz y destruya su capacidad para coordinar el cierre del árbol. Dicho hijo auxiliar ejecuta execlp("pstree", "pstree", "-c", pid_str, NULL) para dibujar la jerarquía en la terminal y finalizar. Por último, el proceso raíz ejecuta un bucle de columnas + 1 llamadas a wait(NULL), recogiendo de forma estricta tanto al proceso hijo encargado de pstree como a los líderes de cada una de las columnas conforme van terminando, garantizando una salida ordenada y evitando la aparición de procesos zombies o huérfanos.

Llamadas al Sistema Utilizadas: sleep(), fork(), execlp(), wait(), exit().

```c
if (soy_raiz) {
    sleep(1);

    char pid_str[16];
    sprintf(pid_str,"%d",pid_raiz);

    if (fork() == 0) {
        execlp("pstree","pstree","-c",pid_str,NULL);
        perror("Error al ejecutar pstree");
        exit(1);
    }

    for (int i = 0; i < columnas + 1; i++) { // me aseguro de esperar a todos los hijos incluso el pstree
        wait(NULL);
    }
```

1.4.5. Bloque de los Procesos Hijos: Espera con wait(), Pausa y Salida

El objetivo de este bloque es coordinar la vida y la finalización ordenada de todos los procesos descendientes que forman las columnas. Mediante la bifurcación condicional, los eslabones intermedios que tienen un hijo debajo (pid > 0) se detienen en una llamada wait(NULL), impidiendo que ningún padre termine antes que su descendiente. Por otro lado, los procesos hoja situados en la base (pid == 0) ejecutan una pausa con sleep(3), actuando como anclas para mantener vivas a todas las columnas el tiempo suficiente para que el comando pstree explore y muestre la estructura completa en la terminal. Cuando transcurren los tres segundos, las hojas finalizan con exit(0), lo que desbloquea en cascada ascendente a cada uno de sus padres inmediatos para que terminen ordenadamente tras recoger a sus hijos, cerrando la jerarquía de abajo hacia arriba de forma limpia.

Llamadas al Sistema Utilizadas: wait(), sleep(), exit().

```c
} else {
    if (pid > 0) {
        wait(NULL);
    } else {
        sleep(3); // sino no salen
    }
    exit(0);
}
```

Capítulo 2

Ejercicio 1.b

2.1. Enunciado del Ejercicio

Realiza un programa llamado ejec.c que reciba un argumento. El programa tendrá que generar el árbol de procesos que se indica y llevar a cabo la funcionalidad que se describe a continuación. El proceso Z, transcurridos los segundos indicados por el argumento, enviará una señal al proceso A. El proceso A, al recibir la señal, ejecutará el comando "pstree". El proceso Z no puede utilizar el comando "sleep", por lo que el proceso Z debe planificarse una alarma con los segundos indicados por el argumento. Se deberá controlar la correcta destrucción del árbol (los padres no pueden morir antes que los hijos).

2.1.1. Estructura del Árbol Solicitada

La jerarquía de procesos que se debe crear es la siguiente:

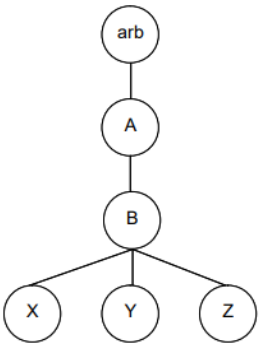


Figura 2.1: Diagrama del árbol de procesos a generar.

2.1.2. Resultado Esperado en la Terminal

A continuación se muestra un ejemplo de la salida esperada por la terminal:

```
$ ejec 15
Soy el proceso ejec: mi pid es 751
Soy el proceso A: mi pid es 752. Mi padre es 751
Soy el proceso B: mi pid es 753. Mi padre es 752. Mi abuelo es 751
Soy el proceso X: mi pid es 754. Mi padre es 753. Mi abuelo es 752. Mi bisabuelo es 751
Soy el proceso Y: mi pid es 755. Mi padre es 753. Mi abuelo es 752. Mi bisabuelo es 751
Soy el proceso Z: mi pid es 756. Mi padre es 753. Mi abuelo es 752. Mi bisabuelo es 751
/*Tras un intervalo de 15 segundos aparecerá*/
/*Resultado del comando "pstree" */
Soy Z (756) y muero
Soy Y (755) y muero
Soy X (754) y muero
Soy B (753) y muero
Soy A (752) y muero
Soy ejec (751) y muero
```

2.2. Identificación y Justificación de Tareas

2.2.1. Recepción y Validación de Argumentos

El enunciado establece: “Realiza un programa llamado ejec.c que reciba un argumento”. La justificación de esta tarea es la necesidad de definir dinámicamente en tiempo de ejecución el intervalo de espera en segundos que deberá temporizar el proceso Z antes de desencadenar la acción principal. Para llevarlo a cabo, la función main() evalúa inicialmente si el número de parámetros recibidos es suficiente mediante la condición argc < 2. En caso de no ser así, deja en la función auxiliar errores(2), la cual notifica el error por consola y finaliza la ejecución con exit(1). Si la validación es buena, se convierte el argumento de texto a un valor numérico entero mediante la función atoi(argv[1]), almacenándolo en la variable segundos. De tal forma, el proceso coordinador consulta y registra su identificador único mediante getpid() en la variable global pid_arb, permitiendo que todos los descendientes conozcan el identificador de la raíz del árbol genealógico.

2.2.2. Generación de la Jerarquía de Procesos

El problema exige de manera explícita: “El programa tendrá que generar el árbol de procesos que se indica”, estableciendo una estructura compuesta por un tronco lineal y un abanico que tendrá tres hojas (X, Y, Z). La justificación técnica radica en crear cada proceso de forma secuencial y controlada, asegurando que cada nodo reciba los identificadores numéricos de sus ancestros directos para cumplir con el formato de presentación por pantalla requerido. La implementación se genera de forma descendente: la función main() bifurca con fork() para dar vida a la función proceso_A(), esta, a su vez, genera a proceso_B() transmitiéndole su propio PID (pid_a). Finalmente, el proceso B ejecuta un bucle iterativo de tres pasos (i = 0, 1, 2) para engendrar mediante fork() a los procesos X, Y y Z, guardando sus identificadores en el vector pids[3]. Nada más nacer, cada proceso imprime en la salida estándar su identificación genealógica completa (padre, abuelo y bisabuelo) antes de pasar a su fase siguiente.

2.2.3. Temporización con Alarma en el Proceso Z (sin usar sleep)

El enunciado impone una restricción muy importante: “El proceso Z no puede utilizar el comando sleep, por lo que el proceso Z debe planificarse una alarma con los segundos indicados por el argumento”. La justificación es forzar el uso de mecanismos de sincronización asíncrona proporcionados por el núcleo del sistema operativo en lugar de recurrir a bloqueos convencionales. Para cumplir esto, dentro de la función proceso_B(), el proceso Z (i = 2) asocia la señal SIGALRM a la rutina vacía despertador() mediante la llamada signal(). Seguidamente, programa la temporización en el reloj del sistema mediante

alarm(segundos) y se suspende de manera pasiva invocando pause(). Al vencer la cuenta, el núcleo emite la señal SIGALRM al proceso Z, lo que interrumpe el estado de hibernación de pause() y le permite continuar con la ejecución de sus instrucciones sin consumir ciclos de CPU durante el intervalo de espera.

2.2.4. Notificación entre Procesos con Señales y Ejecución de pstree

La sincronización se describe mediante: “El proceso Z, transcurridos los segundos indicados por el argumento, enviará una señal al proceso A. El proceso A, al recibir la señal, ejecutará el comando pstree”. La justificación de esta tarea es coordinar la comunicación entre dos procesos situados en niveles jerárquicos distintos sin emplear archivos ni tuberías, delegando además la ejecución de una orden externa sin destruir la imagen del proceso receptor. Al despertar de su alarma, el proceso Z utiliza la llamada kill(pid_a, SIGUSR1) para notificar al proceso A. Dado que el proceso A había preparado previamente el manejador tiempo_pasado() mediante signal(SIGUSR1, tiempo_pasado), la recepción de la señal desencadena automáticamente esta rutina. Dentro del manejador, el proceso A genera un proceso hijo temporal con fork() para que este invoque execlp("pstree", ...) y dibuje el árbol en la terminal, mientras el propio manejador espera su finalización con wait(NULL), preservando así el mapa de memoria y la existencia de A en el sistema.

2.2.5. Destrucción Ordenada del Árbol de Abajo hacia Arriba

El enunciado especifica: “Se deberá controlar la correcta destrucción del árbol (los padres no pueden morir antes que los hijos)”, requiriendo una secuencia de finalización estricta. La justificación es evitar la generación de procesos huérfanos, garantizando que cada proceso padre recolecte a sus propios descendientes antes de abandonar el sistema. Para coordinar este apagado escalonado, tras finalizar la ejecución de pstree, el proceso A envía una señal SIGUSR1 a su hijo B y se bloquea en un wait(NULL). Al ser despertado, el proceso B toma el control de la destrucción e itera en sentido decreciente (i = 2, 1, 0): envía SIGUSR1 sucesivamente a Z, a Y y a X mediante kill(), y tras cada envío ejecuta un wait(NULL). Cada una de las hojas, que permanecía suspendida en un pause(), se desbloquea, imprime su mensaje de despedida y finaliza con exit(0). Cuando los tres hijos han sido recolectados, el proceso B imprime su muerte y finaliza, desbloqueando a su vez al proceso A, que concluye y permite que el proceso raíz ejec finalice de manera ordenada.

2.3. Diagrama de la Jerarquía de Procesos

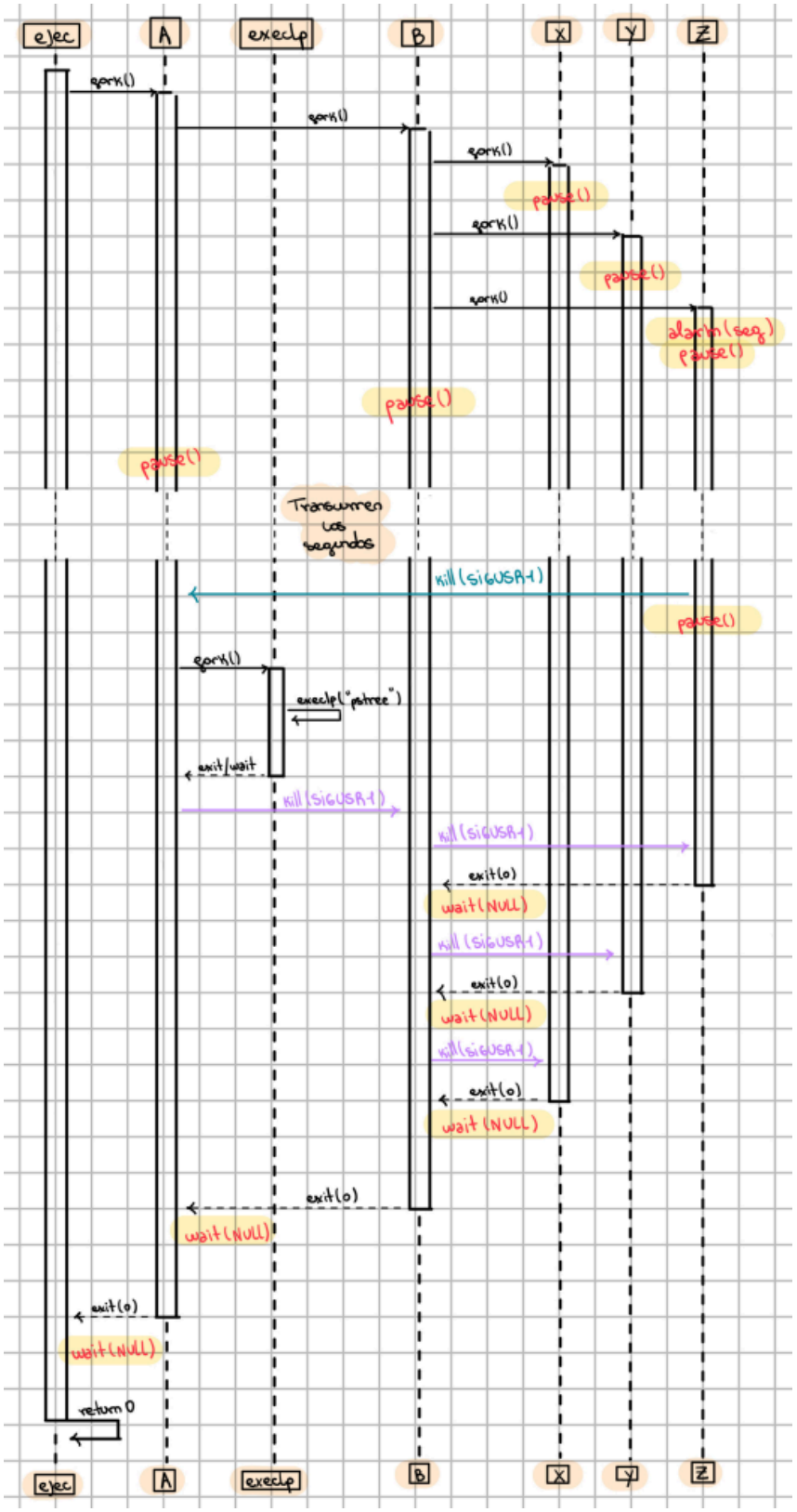


Figura 2.3: Diagrama de secuencia de ejec.c

2.4. Análisis Detallado del Código Fuente

2.4.1. Manejadores de Señal y Funciones Auxiliares (tiempo_pasado, despertador, errores)

Este bloque agrupa las rutinas de soporte para la gestión de errores y el tratamiento asíncrono de señales del sistema operativo. La función errores() centraliza la notificación de incidencias (fallos en la bifurcación o sintaxis incorrecta) y garantiza una salida controlada del programa mediante exit(1). Por su parte, la rutina despertador() actúa como un manejador cuyo único propósito es capturar de forma pasiva las señales SIGUSR1 o SIGALRM, evitando que el sistema operativo aplique la acción por defecto de terminar el proceso y permitiendo que este se desbloquee de su llamada pause() para continuar su flujo de ejecución. Finalmente, la función tiempo_pasado() se ejecuta en el proceso A al recibir la señal de aviso enviada por Z: formatea el identificador de la raíz mediante sprintf() y genera un proceso hijo efímero mediante fork() para delegar en él la sustitución de imagen con execlp("pstree", "pstree", "-c", "-p", pid_str, NULL), mientras dicho comando dibuja la jerarquía completa en pantalla, la rutina espera su finalización mediante wait(NULL), preservando el espacio de memoria y la existencia del proceso A.

Llamadas al Sistema Utilizadas: fork(), execlp(), wait(), exit().

```c
void errores(int tipo) {
    switch (tipo) {
        case 1:
            printf("No he podido crear los procesos\n");
            exit(1);
            break;
        case 2:
            printf("Argumentos incorrectos\n");
            exit(1);
            break;
    }
}

void despertador(int s) {}

void tiempo_pasado(int s) {
    char pid_str[16];
    sprintf(pid_str, "%d", pid_arb);

    if (fork() == 0) {
        execlp("pstree", "pstree", "-c", "-p", pid_str, NULL);
        perror("Error al ejecutar pstree");
        exit(1);
    }
    wait(NULL);
}
```

2.4.2. Función main(): Proceso Coordinador ejec

La función main() constituye el punto de entrada del programa y representa al proceso raíz ejec, actuando como el ancestro común de toda la jerarquía y el supervisor final del ciclo de vida del árbol. En primer lugar, evalúa la presencia del argumento obligatorio mediante la condición argc < 2, invocando a errores(2) en caso de omisión. Tras validar la entrada, consulta su propio identificador con getpid() y lo almacena en la variable global pid_arb para que esté disponible para todos sus descendientes, convierte la cadena de tiempo a un entero mediante atoi(argv[1]) e imprime su mensaje de presentación por pantalla. A continuación, engendra al primer eslabón del tronco mediante fork(): en la rama hija (case 0) cede el control a la función proceso_A(segundos), mientras que en la rama padre (default) se bloquea indefinidamente mediante wait(NULL) a la espera de que toda la descendencia termine. Una vez recolectado su hijo A, imprime su mensaje de muerte y finaliza la aplicación garantizando la muerte de todos los recursos.

Llamadas al Sistema Utilizadas: getpid(), fork(), wait(), exit().

```c
int main(int argc, char *argv[]) {
    if (argc < 2) errores(2);

    pid_arb = getpid();
    int segundos = atoi(argv[1]);

    printf("Soy el proceso ejec: mi pid es %d\n", pid_arb);

    pid_t pid_hijo_a = fork();

    switch (pid_hijo_a) {
        case -1:
            errores(1);
            break;
        case 0:
            proceso_A(segundos);
            break;
        default:
            wait(NULL);
            printf("Soy ejec (%d) y muero\n", pid_arb);
            break;
    }
    return 0;
}
```

2.4.3. Función proceso_A(): Espera de Señal y Disparo de pstree

Esta función encapsula el comportamiento del proceso A, cuya responsabilidad es actuar como receptor de la notificación temporal y disparar la orden de visualización antes de empezar la fase de destrucción. Al comenzar, obtiene su identificador mediante getpid(),

asocia la señal SIGUSR1 al manejador tiempo_pasado mediante signal() y emite su mensaje de nacimiento indicando a su padre pid_arb. Seguidamente, invoca fork() para crear al nodo B: la rama hija entra directamente a ejecutar proceso_B(pid_a, segundos). El proceso A, en su rama padre, se suspende en una llamada pause() a la espera de la señal de Z. Cuando dicha señal llega, se ejecuta el manejador que muestra el árbol con pstree y, al retornar al flujo principal, A notifica el inicio de la destrucción a su hijo B mediante kill(pid_hijo_b, SIGUSR1). Acto seguido, se detiene en un wait(NULL) para esperar la muerte de B, tras lo cual imprime su mensaje de muerte y termina con exit(0).

Llamadas al Sistema Utilizadas: getpid(), signal(), fork(), pause(), kill(), wait(), exit().

```c
void proceso_A(int segundos) {
    int pid_a = getpid();
    signal(SIGUSR1, tiempo_pasado);
    printf("Soy el proceso A: mi pid es %d. Mi padre es %d\n", pid_a, pid_arb);

    pid_t pid_hijo_b = fork();

    switch (pid_hijo_b) {
        case -1:
            errores(1);
            break;
        case 0:
            proceso_B(pid_a, segundos);
            break;
        default:
            pause();
            kill(pid_hijo_b, SIGUSR1);
            wait(NULL);
            printf("Soy A (%d) y muero\n", pid_a);
            exit(0);
    }
}
```

2.4.4. Función proceso_B(): Creación del Abanico (X, Y, Z) y Cascada de Muerte

Esta función gestiona el nodo intermedio B y la vida completa de sus tres procesos descendientes (X, Y y Z). Tras registrar su identificador con getpid(), configurar SIGUSR1 con signal() e imprimir su genealogía completa, el proceso B ejecuta un bucle iterativo de tres pasos para engendrar a las hojas mediante fork() y almacenar sus identificadores en el vector pids[3]. Los procesos X e Y (i=0 e i=1) imprimen sus ascendentes y se suspenden indefinidamente en un pause() aguardando la orden de muerte. El proceso Z (i=2) asocia SIGALRM a despertador(), programa el temporizador con alarm(segundos) y se suspende con pause(), al vencer la alarma, despierta, emite la señal de ejecución hacia A con kill(pid_a, SIGUSR1) y entra en un segundo pause() a la espera de su turno de muerte. Por su parte, el padre B aguarda en un pause() hasta que A le ordena iniciar el apagado. En ese momento, ejecuta un bucle descendente (i = 2, 1, 0) donde despierta ordenadamente a Z, a Y y a X con

kill(pids[i], SIGUSR1) y recolecta inmediatamente a cada uno con wait(NULL) tras su respectivo exit(0). Finalmente, con todas sus hojas destruidas, B imprime su muerte y finaliza con exit(0), permitiendo la conclusión de los ancestros superiores.

Llamadas al Sistema Utilizadas: getpid(), signal(), fork(), pause(), alarm(), kill(), wait(), exit().

```c
void proceso_B(pid_t pid_a, int segundos) {
    int pid_b = getpid();
    pid_t pids[3];
    signal(SIGUSR1, despertador);

    printf("Soy el proceso B: mi pid es %d. Mi padre es %d. Mi abuelo es %d\n", pid_b, pid_a, pid_arb);

    for (int i = 0; i < 3; i++) {
        pids[i] = fork();
        if (pids[i] == -1) errores(1);

        if (pids[i] == 0) { // cada hijo
            signal(SIGUSR1, despertador);

            if (i == 0) {
                printf("Soy el proceso X: mi pid es %d. Mi padre es %d. Mi abuelo es %d. Mi bisabuelo es %d\n", getpid(), pid_b, pid_a, pid_arb);
                pause();
                printf("Soy X (%d) y muero\n", getpid());
                exit(0);
            } else if (i == 1) {
                printf("Soy el proceso Y: mi pid es %d. Mi padre es %d. Mi abuelo es %d. Mi bisabuelo es %d\n", getpid(), pid_b, pid_a, pid_arb);
                pause();
                printf("Soy Y (%d) y muero\n", getpid());
                exit(0);
            } else {
                printf("Soy el proceso Z: mi pid es %d. Mi padre es %d. Mi abuelo es %d. Mi bisabuelo es %d\n", getpid(), pid_b, pid_a, pid_arb);

                signal(SIGALRM, despertador);
                alarm(segundos);
                pause();

                kill(pid_a, SIGUSR1);

                pause();
                printf("Soy Z (%d) y muero\n", getpid());
                exit(0);
            }
        }
    }

    pause();

    for (int i = 2; i > -1; i--) {
        kill(pids[i], SIGUSR1);
        wait(NULL);
    }

    printf("Soy B (%d) y muero\n", pid_b);
    exit(0);
}
```

Capítulo 3

Ejercicio 2

3.1. Enunciado del Ejercicio

3.1.1. Parámetros de Entrada y Esquema de Fragmentación

Realizar un programa llamado hacha.c que divida un archivo en varios trozos con el mismo nombre y extensión h00, h01, … . Se debe tener en cuenta el formato y consideraciones siguientes:

$ hacha <archivo> <tamaño>
<archivo> nombre del archivo a dividir
<tamaño> tamaño en bytes de los archivos divididos

Para dividir, el proceso hacha (proceso padre) generará tantos procesos hijos como archivos se tengan que crear. Mediante tuberías, el proceso padre enviará la información a los hijos y serán estos últimos los que creen los archivos de destino y escriban en ellos.

Consideraciones:

1. Las entradas y salidas a los archivos se realizarán con llamadas al sistema estudiadas en la asignatura (no se puede utilizar printf, scanf, etc.)
2. El padre creará tantos hijos como fragmentos del archivo a realizar, siendo decisión del alumno si los hijos se lanzan secuencial o concurrentemente.

3.1.2. Resultado Esperado en la Terminal

A continuación se muestra un ejemplo de la salida esperada por la terminal:

```
$ hacha at_madrid.mp3 50000
$ ls
at_madrid.mp3.h00
at_madrid.mp3.h01
at_madrid.mp3.h02
```

3.2. Identificación y Justificación de Tareas

3.2.1. Recepción y Validación de Argumentos

El enunciado define la sintaxis de invocación del programa como: “$ hacha <archivo> <tamaño>”, donde se especifican el nombre del archivo a dividir y el tamaño en bytes de los fragmentos resultantes. La justificación de esta tarea es asegurar que el programa cuente con los parámetros obligatorios antes de iniciar cualquier operación sobre el sistema de archivos, evitando desbordamientos o accesos a posiciones inválidas de memoria. Para llevarlo a cabo, la función main() comprueba inicialmente si se han introducido los argumentos necesarios mediante la condición argc < 3. En caso de omisión, notifica el fallo mediante perror("Argumentos incorrectos") y finaliza la ejecución retornando un código de error. Si la validación es correcta, almacena la ruta en nombreArchivo y convierte el segundo argumento a un valor numérico entero mediante la función atoi(argv[2]), guardándolo en la variable bytes para definir el tamaño de los bloques de partición.

3.2.2. Inspección del Tamaño del Archivo y Cálculo de Partes (stat)

El enunciado dice: “Para dividir, el proceso hacha (proceso padre) generará tantos procesos hijos como archivos se tengan que crear”. La justificación viene en que el proceso coordinador debe conocer con antelación el tamaño exacto del fichero en disco para determinar cuántos hijos y fragmentos deberán crearse, sin necesidad de recorrer el archivo ni desplazar sus punteros internos de lectura. La implementación resuelve esto mediante la llamada al sistema stat(nombreArchivo, &info_archivo). Tras verificar que la operación se ejecuta sin errores, el programa extrae el tamaño total en bytes del campo info_archivo.st_size. A partir de este dato, calcula la cantidad de fragmentos mediante la división entera tamaño / bytes, aplicando además un redondeo hacia arriba (cantidad++) si la división presenta un residuo (tamaño % bytes > 0), garantizando así que los bytes sobrantes finales cuenten con su correspondiente hijo y archivo de persistencia.

3.2.3. Canal de Comunicación Mediante Tuberías (pipe) y Gestión de Descriptores

El texto especifica el mecanismo entre procesos: “Mediante tuberías, el proceso padre enviará la información a los hijos y serán estos últimos los que creen los archivos de destino y escriban en ellos”. La justificación técnica radica en emplear un canal de comunicación unidireccional en memoria principal que conecte al padre con cada hijo sin almacenar datos intermedios en disco duro. Para que el flujo de datos opere con normalidad, es obligatorio aislar los descriptores de lectura y escritura no utilizados y cerrar el extremo de escritura del emisor para marcar el fin de datos (EOF) al receptor. El programa invoca la llamada pipe(descriptores) al inicio de cada iteración del bucle. Tras la bifurcación, el padre cierra inmediatamente el extremo de lectura descriptores[0] con close(), escribe los datos

correspondientes en descriptores[1] y procede a cerrarlo en el acto, mientras que el hijo cierra su extremo de escritura descriptores[1] para leer exclusivamente a través de descriptores[0].

3.2.4. Estrategia de Procesamiento Secuencial y Sincronización con wait()

El enunciado ofrece flexibilidad en el diseño: “siendo decisión del alumno si los hijos se lanzan secuencial o concurrentemente”. La justificación de adoptar una aproximación secuencial es evitar condiciones de carrera en las que múltiples procesos hijos compitan por leer de la misma tubería o consuman excesivos descriptores de archivo de forma simultánea en la tabla del proceso. Para garantizar un control estricto, el padre ejecuta un bucle iterativo que atiende a un único hijo por fragmento: tras meter el bloque en la tubería y cerrar la escritura, el padre invoca wait(NULL) y queda suspendido pasivamente hasta que el hijo en curso finaliza su labor con exit(0). Este mecanismo escalonado garantiza que ningún padre muera antes que sus descendientes y asegura la consolidación en disco de cada fragmento antes de procesar el siguiente.

3.2.5. Formateo Manual de Nombres sin Funciones de Alto Nivel

El problema exige que los archivos resultantes tengan: “el mismo nombre y extensión h00, h01, …”, imponiendo como restricción: “no se puede utilizar printf, scanf, etc.”. La justificación de esta tarea es marcar la prohibición estricta de la biblioteca estándar de formateo de texto, requiriendo construir las cadenas de caracteres mediante transformaciones directas en memoria para evitar el uso de funciones de alto nivel. Para implementar esto sin recurrir a sprintf(), cada proceso hijo utiliza strcpy() para copiar la ruta base en nombre_trozo y le concatena el prefijo strcat(nombre_trozo, ".h"). Seguidamente, determina los dos dígitos numéricos mediante aritmética de caracteres ASCII: calcula las decenas sumando '0' + (i / 10) y las unidades con '0' + (i % 10), inserta el delimitador nulo \0 en el vector auxiliar extension y lo une mediante strcat(), generando extensiones exactas de dos cifras (.h00, .h01, etc.) sin dañar lo demandado de la práctica.

3.2.6. Transferencia de Bloques Binarios en Bajo Nivel (read y write)

El enunciado impone: “Las entradas y salidas a los archivos se realizarán con llamadas al sistema estudiadas en la asignatura”. La justificación de esta decisión es garantizar que la fragmentación sea transparente al tipo de contenido del archivo, preservando bien ficheros binarios complejos (como imágenes, ejecutables o audio MP3) donde funciones de texto se truncarían al encontrar bytes nulos intermedios. Para lograrlo, el padre abre el archivo original una única vez al inicio mediante open(nombreArchivo, O_RDONLY). En cada iteración, el padre extrae hasta bytes con read(df_origen, informacion, bytes) y transfiere exactamente los bytes leídos al tubo con write(descriptores[1], informacion, leidos). A su vez,

el hijo crea el archivo de destino con open(nombre_trozo, O_WRONLY | O_CREAT | O_TRUNC, 0666), extrae los datos del canal con read(descriptores[0], buffer_hijo, bytes) y los persiste con write(df_destino, buffer_hijo, leidos_tubo). Finalmente, ambos procesos liberan los descriptores involucrados con close(), completando una transferencia binaria limpia.

3.3. Diagrama de la Jerarquía de Procesos

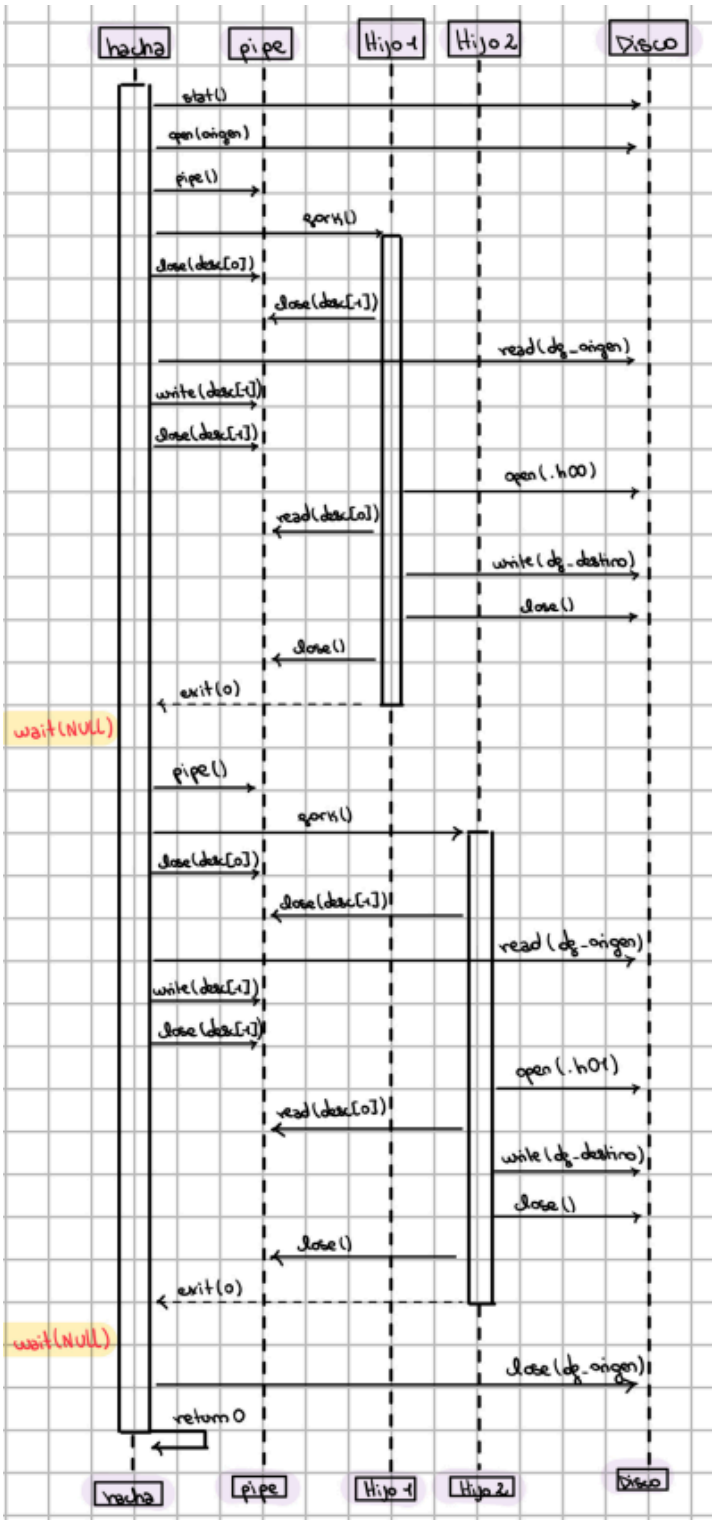


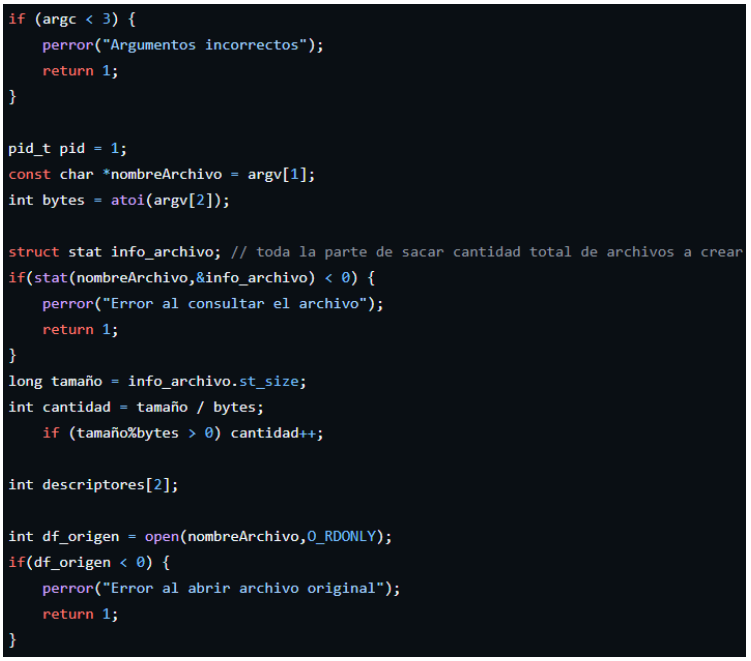
Figura 3.3: Diagrama de secuencia de hacha.c

3.4. Análisis Detallado del Código Fuente

3.4.1. Bloque de Validación y Apertura Inicial (stat, open)

El propósito de este bloque es asegurar la integridad de los parámetros introducidos por la línea de comandos, inspeccionar las propiedades del archivo de entrada y dejar preparado su acceso antes de comenzar la creación de procesos. En primer lugar, se evalúa mediante la condición argc < 3 que el usuario haya especificado la ruta del archivo y el tamaño de bloque, deteniendo la ejecución con código de error si faltan argumentos. Tras convertir la cadena del tamaño a entero mediante atoi(argv[2]), se invoca la llamada stat() sobre el fichero original para consultar sus datos sin necesidad de leerlo: a partir del campo info_archivo.st_size, se calcula el número de fragmentos necesarios dividiendo el tamaño total entre el tamaño del bloque (tamaño / bytes), incrementando la variable cantidad en una unidad si existe residuo (tamaño % bytes > 0) para garantizar que el resto final disponga de su propio proceso. Finalmente, el proceso padre abre el archivo original una sola vez mediante open(nombreArchivo, O_RDONLY), obteniendo el descriptor df_origen que mantendrá la posición del cursor de lectura sincronizada a lo largo de todas las iteraciones.

Llamadas al Sistema Utilizadas: stat(), open().



```
if (argc < 3) {
    perror("Argumentos incorrectos");
    return 1;
}

pid_t pid = 1;
const char *nombreArchivo = argv[1];
int bytes = atoi(argv[2]);

struct stat info_archivo; // toda la parte de sacar cantidad total de archivos a crear
if(stat(nombreArchivo,&info_archivo) < 0) {
    perror("Error al consultar el archivo");
    return 1;
}
long tamaño = info_archivo.st_size;
int cantidad = tamaño / bytes;
    if (tamaño%bytes > 0) cantidad++;

int descriptores[2];

int df_origen = open(nombreArchivo,O_RDONLY);
if(df_origen < 0) {
    perror("Error al abrir archivo original");
    return 1;
}
```

3.4.2. Bloque del Proceso Padre: Lectura de Bloques, Envío por Tubería y Espera

Este bloque define el rol del proceso coordinador dentro del bucle de partición, encargándose de generar secuencialmente los canales de comunicación y transferir los datos a cada proceso

hijo. En cada una de las iteraciones, el padre crea un canal de comunicación unidireccional con pipe(descriptores) y genera un nuevo proceso mediante fork(). Al identificarse como padre (pid > 0), procede a cerrar inmediatamente el descriptor de lectura de la tubería descriptores[0] con close(), ya que su propósito es exclusivamente emisor. A continuación, extrae del archivo original un bloque de hasta bytes mediante read(df_origen, informacion, bytes) y transfiere exactamente los bits leídos hacia el canal con write(descriptores[1], informacion, leidos). Tras volcar la información, cierra de inmediato el extremo de escritura descriptores[1], lo que señaliza al proceso hijo que no se transmitirán más datos y provoca que su posterior lectura devuelva el fin de archivo (EOF). Por último, el padre invoca wait(NULL) para quedar suspendido hasta que el hijo en curso finalice la creación y escritura de su fragmento, repitiendo este ciclo hasta completar todos los bloques y cerrando finalmente el archivo fuente con close(df_origen).

Llamadas al Sistema Utilizadas: pipe(), fork(), close(), read(), write(), wait().

```
if (pid > 0) {
    close(descriptores[0]);

    char informacion[bytes];
    int leidos = read(df_origen,informacion,bytes);

    if (write(descriptores[1],informacion,leidos) < 0) {
        perror("Error en padre");
    }

    close(descriptores[1]);
    wait(NULL);
```

3.4.3. Bloque del Proceso Hijo: Creación del Trozo, Recepción y Persistencia

Este bloque tiene la lógica que ejecuta cada uno de los procesos hijos dentro de la rama else (pid == 0), teniendo como única responsabilidad fabricar su fragmento correspondiente y volcar en él los datos procedentes de la tubería. En primer lugar, el hijo cierra el extremo de escritura de la tubería descriptores[1] para dedicarse únicamente a recibir información. A continuación, construye el nombre del archivo destino sin emplear funciones de formateo prohibidas: utiliza strcpy() y strcat() para fijar el nombre base con la extensión .h, y calcula mediante aritmética de caracteres ASCII la cifra de las decenas ('0' + (i / 10)) y de las unidades ('0' + (i % 10)), completando la cadena terminada en \0 para dar lugar a nombres como .h00 o .h01. Seguidamente, crea y abre el fichero resultante en disco mediante open() con las banderas O_WRONLY | O_CREAT | O_TRUNC y permisos octales 0666. Acto seguido, drena los datos presentes en el canal mediante read(descriptores[0], buffer_hijo,

bytes) y los escribe íntegramente en el archivo destino con write(df_destino, buffer_hijo, leidos_tubo). Para finalizar de forma limpia, el hijo cierra su archivo creado, el descriptor de lectura de la tubería y el descriptor heredado del archivo original mediante tres llamadas a close(), concluyendo su ejecución con exit(0) para desbloquear la espera de su padre.

Llamadas al Sistema Utilizadas: close(), open(), read(), write(), exit().

```
} else {
    close(descriptores[1]);
    char buffer_hijo[bytes];
    char nombre_trozo[256];
    char extension[3];

    strcpy(nombre_trozo,nombreArchivo);
    strcat(nombre_trozo,".h");

    extension[0] = '0' + (i/10); // decenas
    extension[1] = '0' + (i%10); // unidades
    extension[2] = '\0'; // terminador

    strcat(nombre_trozo,extension);

    int df_destino = open(nombre_trozo, O_WRONLY | O_CREAT | O_TRUNC, 0666);
    if (df_destino < 0) {
        perror("Error al crear el trozo hijo");
        exit(1);
    }

    int leidos_tubo = read(descriptores[0],buffer_hijo, bytes);
    write(df_destino, buffer_hijo,leidos_tubo);

    close(df_destino);
    close(descriptores[0]);
    close(df_origen);
    exit(0);
}
```

Capítulo 4

Ejercicio 3

4.1. Enunciado del Ejercicio

Diseña un programa llamado hijos.c que cree un árbol de procesos según la siguiente estructura a partir de dos parámetros *$ hijos x y*.

El proceso hijos debe mostrar el siguiente mensaje:
“Soy el superpadre (pid) : mis hijos finales son: pidx1, pidx2, pidx3, …, pidxy”.

Los procesos de la parte baja deben mostrar el siguiente mensaje:
“Soy el subhijo pidx1, mi padres son: pid1, pid2, …, pidx”.

4.1.1. Estructura del árbol (Tronco y Abanico)

La jerarquía de procesos que se debe crear es la siguiente:

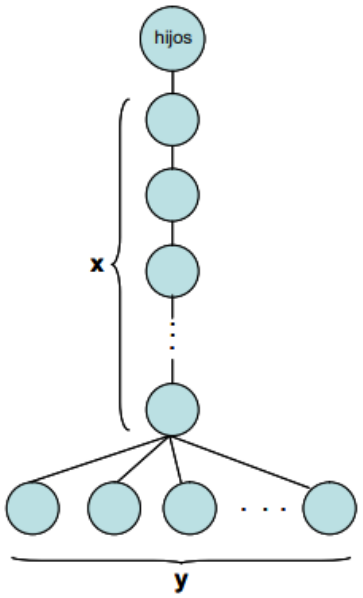


Figura 4.1: Diagrama del árbol de procesos a generar.

4.2. Identificación y Justificación de Tareas

4.2.1. Recepción y Validación de Argumentos

El enunciado establece que: “a partir de dos parámetros *$ hijos x y*”, donde x representa la altura de la cadena vertical e y el número de procesos hoja terminales. La justificación de esta tarea es garantizar que la aplicación cuente con las dimensiones requeridas para dimensionar las estructuras en memoria y controlar los bucles de bifurcación antes de iniciar la creación del árbol. Para llevarlo a cabo, la función main() evalúa inicialmente si se han introducido los argumentos obligatorios mediante la condición argc < 3. En caso de omisión, se notifica el fallo mediante perror("Argumentos incorrectos") y se aborta la ejecución retornando código de error. Si la validación es satisfactoria, se convierten los argumentos de texto a enteros mediante la función atoi(), almacenándolos en las variables longitud_x y longitud_y.

4.2.2. Necesidad y Configuración del Segmento de Memoria Compartida (shmget, shmat)

El enunciado exige que tanto el superpadre como los subhijos finales muestren mensajes cruzados con los identificadores de procesos de niveles opuestos del árbol. La justificación técnica radica en que, al realizar llamadas fork(), el sistema operativo duplica el espacio de direcciones de forma totalmente aislada e independiente para cada proceso, impidiendo que las modificaciones en variables normales sean visibles entre ancestros y descendientes. Para evitar este aislamiento de memoria sin emplear flujos secuenciales de tuberías, se debe reservar un segmento IPC de memoria compartida en la RAM del sistema. La implementación calcula el tamaño exacto en bytes mediante (longitud_x + longitud_y) * sizeof(pid_t) y solicita el recurso mediante shmget(IPC_PRIVATE, tamaño, IPC_CREAT | 0666). Seguidamente, se vincula el segmento al espacio de memoria del proceso mediante tabla = (pid_t *) shmat(shmid, NULL, 0), obteniendo un puntero compartido que actúa como un vector global donde las primeras X posiciones se reservan para los ancestros y las siguientes Y para los subhijos.

4.2.3. Construcción del Tronco Vertical de Ancestros sin Ruptura Estructurada

El diseño exige formar una columna vertical de x niveles de profundidad donde cada eslabón conozca y registre a su progenitor. La justificación radica en encadenar una sucesión estrictamente lineal de procesos evitando bifurcaciones no deseadas. Para resolverlo, el superpadre registra inicialmente su identificador en tabla[0] = getpid(). A continuación, se ejecuta un bucle condicionado a i < longitud_x - 1 && pid == 0. Cuando un proceso invoca fork(), el proceso padre recibe el PID positivo del hijo (pid > 0), lo que invalida de inmediato la condición de permanencia y lo expulsa del bucle de forma natural. El hijo recién nacido, al recibir pid == 0, avanza a la siguiente iteración, desactiva la bandera soy_superpadre = 0, anota su propio identificador en tabla[i + 1] y continúa la secuencia hasta alcanzar el último nivel del tronco.

4.2.4. Creación del Abanico Horizontal de Subhijos y Registro Concurrente de PIDs

El enunciado especifica: “Los procesos de la parte baja deben mostrar el siguiente mensaje: 'Soy el subhijo pidx1, mi padres son: pid1, pid2, ... pidx'”. La justificación técnica es que únicamente el último padre del tronco vertical (pid_x) debe actuar como emisor horizontal de los y subhijos, permitiendo que cada uno registre su propia identidad y consulte concurrentemente el historial de sus ancestros. La implementación restringe el acceso a este bloque mediante if (pid == 0) y ejecuta un bucle controlado por j < longitud_y && pid_subhijo > 0. En cada vuelta, el padre crea a un nuevo hermano mediante fork(). Cada subhijo (pid_subhijo == 0) anota su identificador en la sección de hojas de la memoria compartida mediante tabla[longitud_x + j] = getpid(), recorre las primeras longitud_x casillas para imprimir por consola a todos sus antecesores y finaliza inmediatamente con exit(0), garantizando que no engendre descendencia adicional.

4.2.5. Sincronización Ascendente con wait() y Terminación Limpia

Al igual que en los ejercicios anteriores, rige el principio de que los procesos padre no deben finalizar antes que sus hijos, garantizando que el sistema no genere procesos zombies ni delegue huérfanos. Además, en este caso la justificación se extiende a la consistencia de los datos: el superpadre no puede proceder a la lectura de los resultados hasta que todos los subhijos hayan escrito sus PIDs en la memoria compartida. La implementación coordina esta sincronización en dos fases: en primer lugar, el último padre del tronco ejecuta un bucle de Y llamadas a wait(NULL) para esperar la terminación de todos sus subhijos antes de morir con exit(0). En segundo lugar, cada uno de los padres del tronco vertical aguarda en su correspondiente wait(NULL) y termina con exit(0) si !soy_superpadre. Esta cascada de desbloqueos escalonados asegura que el superpadre sea el último en despertar, garantizando que la tabla contenga la información completa antes de su lectura.

4.2.6. Liberación y Destrucción del Segmento IPC (shmdt, shmctl)

El enunciado propone finalmente: “El proceso hijos debe mostrar el siguiente mensaje: 'Soy el superpadre (pid): mis hijos finales son: ...'”. La justificación radica en completar el flujo de información y realizar una liberación responsable de los recursos del núcleo, puesto que los segmentos de memoria compartida permanecen alojados en el kernel incluso después de que los procesos mueran si no se solicita expresamente su eliminación. Una vez desbloqueado de su wait(NULL), el superpadre (soy_superpadre == 1) lee las posiciones longitud_x a longitud_x + longitud_y - 1 de la memoria compartida e imprime la lista entera de sus subhijos finales. Seguidamente, desvincula el puntero del espacio de direcciones del proceso invocando shmdt((char *)tabla) y destruye definitivamente el recurso en el núcleo del sistema operativo mediante la llamada shmctl(shmid, IPC_RMID, 0).

4.3. Diagrama de la Jerarquía de Procesos

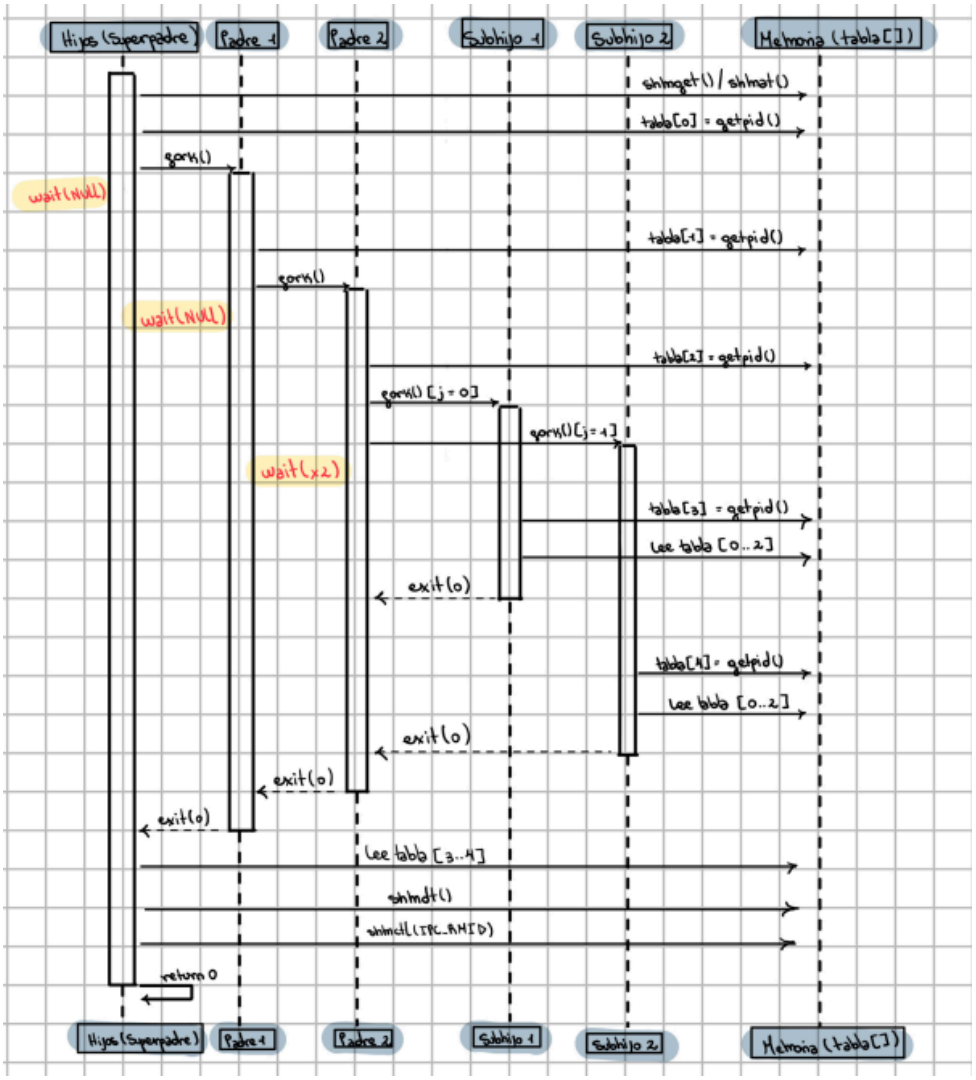


Figura 4.3: Diagrama de secuencia de hijos.c

4.4. Análisis Detallado del Código Fuente

4.4.1. Bloque de Validación, Creación y Vinculación de Memoria IPC

El propósito de este bloque inicial es procesar los argumentos recibidos por la línea de comandos, dimensionar el recurso compartido y dejar acoplado el segmento de memoria en el espacio de direcciones del proceso superpadre antes de iniciar cualquier bifurcación. Para ello, el programa comprueba mediante la condición argc < 3 que se hayan introducido las dimensiones X e Y, finalizando con error mediante perror() en caso contrario. Tras convertir ambos parámetros a valores numéricos con atoi(), se calcula el tamaño total en bytes necesario para almacenar (X + Y) identificadores multiplicando por sizeof(pid_t). Con esta dimensión, se solicita al núcleo la creación de un nuevo segmento IPC mediante shmget(IPC_PRIVATE, tamaño, IPC_CREAT | 0666). Una vez obtenido el identificador shmid, la llamada shmat(shmid, NULL, 0) vincula el segmento devolviendo un puntero que se moldea a pid_t *tabla, permitiendo tratar la memoria compartida como un vector convencional. Finalmente, se inicializan las variables de control (pid = 0, pid_subhijo = 1 y soy_superpadre = 1) y el superpadre registra su propio identificador en la primera posición del segmento mediante tabla[0] = getpid(), estableciendo el primer eslabón del linaje.

Llamadas al Sistema Utilizadas: shmget(), shmat(), getpid().

```
if (argc < 3) {
    perror("Argumentos incorrectos");
    return 1;
}

int longitud_x = atoi(argv[1]);
int longitud_y = atoi(argv[2]);

size_t tamaño = (longitud_x + longitud_y) * sizeof(pid_t);
int shmid = shmget(IPC_PRIVATE, tamaño, IPC_CREAT | 0666);
if (shmid < 0) {
    perror("Error en memoria compartida");
    return 1;
}

pid_t pid = 0;
pid_t *tabla = (pid_t *) shmat(shmid, NULL, 0);
pid_t pid_subhijo = 1;
int soy_superpadre = 1;
tabla[0] = getpid(); // el superpadre es el primero
```

4.4.2. Bloque del Tronco Vertical: Registro de Ancestros y Descenso Lineal

La finalidad de este bloque es generar la cadena vertical de x ancestros en fila india, garantizando un flujo estructurado sin utilizar sentencias de ruptura como break y coordinando la espera escalonada de los progenitores. El bucle for avanza mientras se cumpla la condición i < longitud_x - 1 && pid == 0. Cada vez que se ejecuta la llamada fork(), el proceso progenitor recibe el PID positivo del hijo (pid > 0), lo que rompe la condición de permanencia y lo expulsa del bucle de forma natural. El proceso hijo recién creado, en cambio, recibe pid == 0, avanza a la siguiente iteración, desactiva la bandera marcando soy_superpadre = 0 y anota su propio identificador en la celda contigua mediante tabla[i + 1] = getpid(), prolongando la cadena hasta el eslabón X. A continuación, todos los procesos que actuaron como padres en el tronco (pid > 0) quedan suspendidos en una llamada wait(NULL) esperando la finalización de su respectivo hijo directo, cuando el hijo inferior concluye, si se trata de un padre intermedio (!soy_superpadre) finaliza inmediatamente con exit(0), deshaciendo el tronco en orden ascendente y manteniendo vivo únicamente al superpadre.

Llamadas al Sistema Utilizadas: fork(), getpid(), wait(), exit().

```
for (int i = 0; i < longitud_x - 1 && pid == 0; i++) {
    pid = fork();
    if (pid == 0) {
        soy_superpadre = 0;
        tabla[i + 1] = getpid(); // cada hijo anota su pid en la siguiente casilla
    }
}

if (pid > 0) {
    wait(NULL);

    if (!soy_superpadre) {
        exit(0);
    }
}
```

4.4.3. Bloque del Abanico de Subhijos: Registro y Lectura de Linaje

Este bloque encapsula el comportamiento del último proceso del tronco vertical (pid_x, identificado de forma correcta porque pid == 0), cuya misión es engendrar en abanico a los y subhijos terminales y esperar su conclusión. El proceso ejecuta un bucle condicionado a j < longitud_y && pid_subhijo > 0. En cada iteración, invoca fork(): el padre permanece en el bucle al recibir un valor positivo y continúa creando hermanos, mientras que cada nuevo subhijo recibe pid_subhijo == 0 y accede a su bloque de ejecución. En él, anota su propio identificador en la sección terminal del segmento compartido mediante tabla[longitud_x + j] = getpid(), recorre las primeras longitud_x casillas para imprimir su mensaje mostrando a toda su línea de ancestros por la consola y termina de inmediato con exit(0) sin generar descendencia adicional. Una vez completada la creación de los Y hermanos, el último padre del tronco ejecuta un bucle de recolección con Y llamadas a wait(NULL) para aguardar la

muerte de todos sus subhijos, tras lo cual finaliza con exit(0) para propagar el desbloqueo hacia el nivel superior del árbol.

Llamadas al Sistema Utilizadas: fork(), getpid(), wait(), exit().

```
if (pid == 0) {
    for (int j = 0; j < longitud_y && pid_subhijo > 0; j++) {
        pid_subhijo = fork();

        if (pid_subhijo == 0) {
            tabla[longitud_x + j] = getpid();

            printf("Soy el subhijo %d, mis padres son: ", getpid());
            for (int n = 0; n < longitud_x; n++) {
                printf("%d%s", tabla[n], (n == longitud_x - 1) ? "" : ", ");
            }
            printf("\n");

            exit(0);
        }
    }

    for (int j = 0; j < longitud_y; j++) { // ultimo padre espera
        wait(NULL);
    }
    exit(0);
}
```

4.4.4. Bloque del Superpadre: Lectura Final, Muerte y Destrucción IPC

Este bloque final es ejecutado exclusivamente por el proceso raíz coordinador (soy_superpadre == 1) una vez que se desbloquea de su llamada wait(NULL), momento en el que toda la descendencia inferior ha terminado y los datos se hallan consolidados en la memoria compartida. Su propósito es mostrar el mensaje de resumen del sistema y liberar definitivamente los recursos asignados por el núcleo. El superpadre imprime su presentación por la salida estándar y ejecuta un bucle que recorre las posiciones longitud_x hasta longitud_x + longitud_y - 1 de tabla, imprimiendo la lista entera de los PIDs de sus subhijos finales. Seguidamente, invoca la llamada shmdt((char *)tabla) para desvincular el segmento de memoria compartida de su espacio de direcciones. Finalmente, destruye la estructura en el núcleo del sistema operativo invocando shmctl(shmid, IPC_RMID, 0), garantizando que el segmento IPC no permanezca ocupando memoria RAM tras el cierre del programa y permitiendo una terminación limpia mediante return 0.

Llamadas al Sistema Utilizadas: getpid(), shmdt(), shmctl().

```
if (soy_superpadre) {
    printf("Soy el superpadre (%d): mis hijos finales son: ", getpid());
    for (int j = 0; j < longitud_y; j++) {
    printf("%d%s", tabla[longitud_x + j], (j == longitud_y - 1) ? "" : ", ");
    }
    printf("\n");

    shmdt((char *)tabla);
    shmctl(shmid, IPC_RMID, 0);
}

return 0;
```

Capítulo 5

5.1. Comparativa entre los mecanismos de comunicación utilizados (Señales vs. Tuberías vs. Memoria Compartida)

A lo largo del desarrollo de la práctica se han implementado y analizado los tres mecanismos fundamentales de comunicación entre procesos (IPC) que proporciona el sistema operativo UNIX: señales, tuberías y memoria compartida. Cada uno de ellos responde a necesidades y compromisos de diseño claramente diferenciados:

Señales (Signals): Operan como interrupciones por software de carácter asíncrono. Resultan la herramienta idónea para la notificación inmediata de eventos puntuales, alarmas temporales (SIGALRM) o sincronización de estados básicos (SIGUSR1), permitiendo además suspender procesos de forma pasiva con pause() para no consumir ciclos de CPU mientras esperan una orden. Sin embargo, presentan limitaciones notables: no permiten transmitir flujos complejos de datos (solo el identificador numérico de la señal), carecen de memoria si se reciben señales idénticas consecutivas y su entrega asíncrona puede interrumpir el flujo normal de ejecución de manera imprevista si no se gestionan adecuadamente los manejadores.

Tuberías (Pipes): Constituyen canales unidireccionales en memoria principal que siguen una política estricta FIFO (First In, First Out). Son el mecanismo idóneo para transferir flujos de datos en crudo (como la división y reconstrucción de ficheros binarios en hacha.c), apoyándose en las llamadas al sistema estándar de entrada/salida read() y write(). Su principal ventaja viene en su sincronización implícita: la lectura se bloquea automáticamente si el canal está vacío y se detecta el fin de fichero (EOF) en cuanto el emisor cierra el extremo de escritura. Por el contrario, su uso está restringido a procesos con parentesco (en tuberías sin nombre), no admiten acceso aleatorio como lseek() y requieren crear dos tuberías independientes si se necesita una comunicación bidireccional.

Memoria Compartida: Es el mecanismo más rápido y eficiente, ya que el núcleo mapea una región de memoria RAM física directamente en el espacio de direcciones de múltiples procesos. Esto permite la lectura y escritura instantánea sobre variables o matrices compartidas (como la tabla de identificadores en hijos.c) sin sobrecoste de copias intermedias ni llamadas al sistema adicionales tras su vinculación inicial. Sin embargo, carece de mecanismos automáticos de sincronización: los procesos no se bloquean de forma nativa al leer o escribir, por lo que es necesario coordinar los tiempos de acceso mediante jerarquías de wait() o mecanismos de sincronización para evitar lecturas de datos incompletos o inconsistentes.

5.2. Consideraciones de diseño tenidas en cuenta: prevención de procesos zombies/huérfanos y gestión responsable de recursos del núcleo

El desarrollo de software a nivel de sistema exige un control importante sobre el ciclo de vida de los procesos y sobre los recursos finitos que gestiona el núcleo:

Cuando un proceso hijo termina con exit(), su estructura permanece retenida en la tabla de procesos del núcleo hasta que su padre directo recoge su estado de finalización mediante wait(). Si el padre no invoca wait(), el hijo se transforma en un proceso zombie, agotando entradas en dicha tabla. Por otro lado, si un proceso padre muere antes que sus hijos, estos quedan huérfanos, siendo adoptados por el proceso inicial del sistema. En los programas desarrollados (malla.c, ejec.c, hacha.c e hijos.c), este problema se ha resuelto mediante una estrategia de destrucción en cascada de abajo hacia arriba: cada padre espera activamente a sus hijos mediante llamadas bloqueantes a wait() antes de ejecutar su propio exit(), garantizando que todos los descendientes sean recolectados y que la consola recupere el control únicamente cuando el árbol ha desaparecido por completo del sistema.

La tabla de descriptores de cada proceso es limitada. En el uso de tuberías y archivos abiertos, mantener abiertos extremos de comunicación innecesarios acarrea fugas de descriptores y bloqueos indefinidos. Por ello, he seguido la buena práctica de cerrar de inmediato los descriptores no utilizados (close(descriptores[0]) en emisores y close(descriptores[1]) en receptores). Esta acción es imprescindible, ya que el cierre deliberado del extremo de escritura es la única señal que permite al lector detectar la condición de fin de archivo (EOF) y desbloquear su ejecución sin quedar suspendido de forma infinita.

A diferencia de la memoria local de un proceso, que se libera automáticamente al terminar, los segmentos de memoria compartida creados con shmget() persisten en el núcleo del sistema operativo de forma indefinida incluso después de que todos los procesos hayan finalizado. Si no se destruyen expresamente, continúan consumiendo memoria física en el computador. En el programa hijos.c, se garantiza el ciclo de vida completo del recurso: cada proceso desvincula el segmento de su espacio de memoria mediante shmdt(), y el proceso superpadre ejecuta finalmente la llamada shmctl(shmid, IPC_RMID, 0) para ordenar su destrucción física en el núcleo, asegurando que el sistema quede totalmente limpio tras su ejecución.